JOUR 3

Orchestrer les services

Mercredi 21 octobre 2026 - 9h10-17h10 - Infrastructure, orchestration et automatisation

Mastère Data Engineering & IA, M2 - EFREI Bordeaux - Alexandre STOPPINI

AU PROGRAMME

- Décrire chaque service de VéloCité pour Kubernetes, une fois pour tous les environnements
- Faire tourner dev, puis prod, avec les garde-fous de mardi
- Livrer une version qui casse, sans couper le service, et revenir en arrière

JOUR 3

Plan de la journée

390 minutes utiles - deux modules - deux ateliers - fin sur un atelier

Créneau	Bloc	Contenu
09h10 - 09h30	QCM	Consolidation de la journée 2, puis le dépôt du jour
09h30 - 10h45	Module 5	Décrire un service pour Kubernetes
10h45 - 11h00	Pause	
11h00 - 12h40	Atelier 5	VéloCité dans l'environnement dev, puis le point de midi
12h40 - 13h40	Déjeuner	
13h40 - 14h25	Module 6	Mettre à jour sans couper, revenir en arrière
14h25 - 16h55	Atelier 6	La prod, puis une nouvelle version (pause 14h55 - 15h10)
16h55 - 17h10	Bilan	La preuve du jour, ce qui arrive demain

JOUR 3

Le QCM, puis le dépôt du jour

15 minutes de QCM, non noté - puis les fichiers du jour, chez les deux membres du binôme

Le QCM

- Lien sur la page Moodle du module
- Seul, sans document
- Après l'envoi : "Afficher le score", lisez les justifications
- Ne compte pas dans la note

Windows, macOS et Linux - depuis $DEPOT

```
git pull --no-rebase --no-edit cours j3
# complement_J3.zip (Moodle), décompressé à la
# racine du dépôt, en remplaçant les fichiers ;
# remplir les A_REMPLIR, tofu validate, puis :
git add -A ; git commit -m "Atelier 4 : valeurs"
git push
kubectl config use-context kind-velocite
kubectl get nodes
# Dès le début de l'atelier 5 :
./scripts/charger-images.ps1
# macOS, Linux : pwsh ./scripts/charger-images.ps1
```

MODULE 5

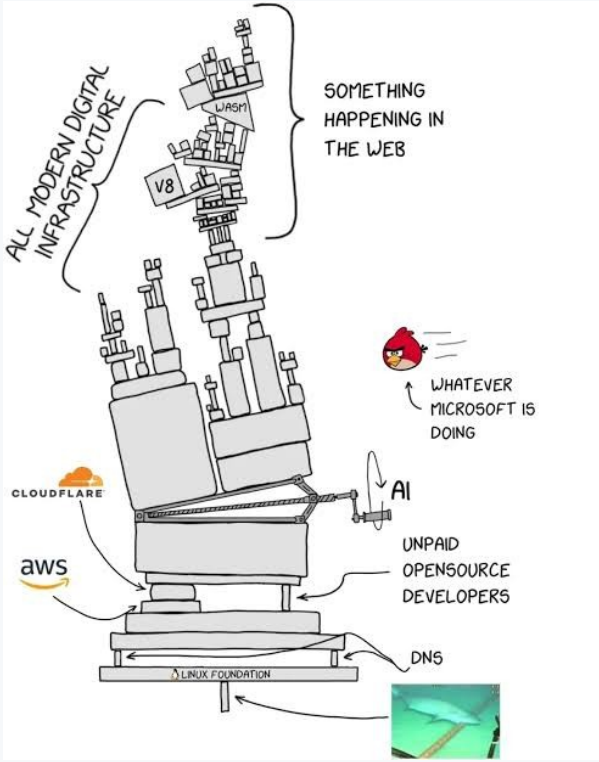
Décrire un service pour Kubernetes

- Ce qui se passe quand on applique un manifeste, du poste au conteneur
- Le modèle api.yaml, bloc par bloc : étiquettes, sécurité, secrets
- Une base et des overlays : Kustomize ; les images dans kind ; le réseau

MODULE 5

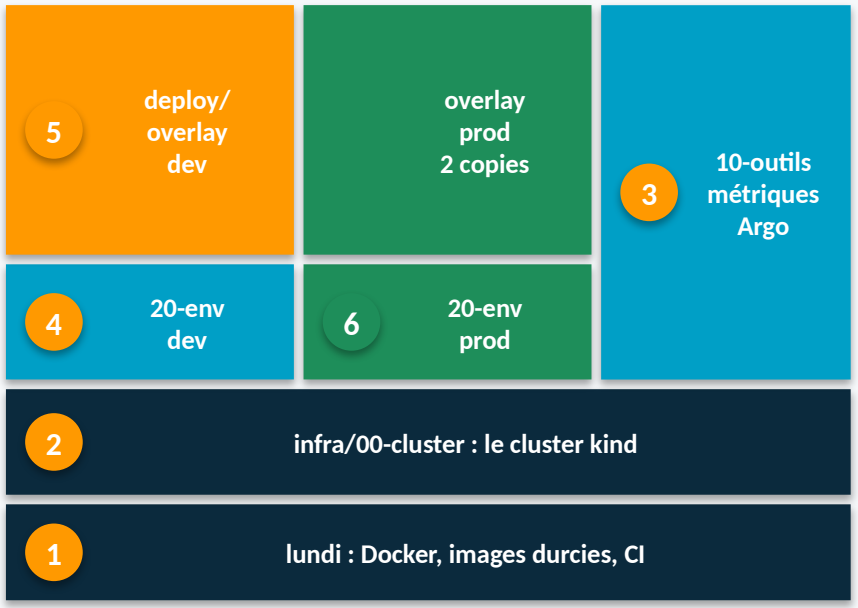
Ce sur quoi tout repose

Chaque couche de la semaine porte la suivante : aujourd'hui, on pose les applications, puis la prod



D'après xkcd 2347, Randall Munroe (CC BY-NC 2.5), version détournée

7 Comme sur le dessin, tout tient sur de petites pièces. Chez nous : le cluster, et la phrase du state de 20-env. Un -/+ sur l'un, et tout tombe.



6 Cet après-midi : la prod, à côté, sur le même socle

5 Ce matin : les manifestes, VéloCité dans dev

4 Mardi : dev, un namespace fermé, ses secrets

3 Mardi : les outils du cluster

2 Mardi : le cluster, décrit en code

1 Lundi : des images sûres, vérifiées par la CI

MODULE 5

Les mots d'abord

Huit mots pour lire un manifeste, une ligne chacun

Mot	Ce que c'est, en une ligne
Manifeste	Un fichier YAML qui décrit un objet : son type (kind), son nom, ce qu'on veut (spec)
Pod	Un ou plusieurs conteneurs lancés ensemble, sur un même node, avec une adresse IP
Deployment	Garde N copies identiques d'un pod en marche ; les remplace si elles tombent
ReplicaSet	Les copies d'une version du pod : une nouvelle version, un nouveau ReplicaSet
Étiquette, sélecteur	Une paire clé: valeur posée sur un pod ; un sélecteur choisit les pods qui l'ont
Service	Un nom fixe (api) qui envoie le trafic aux pods choisis par son sélecteur
Sonde (probe)	Un test répété par Kubernetes : démarré ? prêt à recevoir ? toujours vivant ?
requests / limits	Ce qu'un conteneur réserve au départ / le plafond qu'il ne peut pas dépasser

MODULE 5

Ce qui se passe quand vous appliquez

Suivez le manifeste : six étapes entre votre commande et un conteneur qui répond

```
kubectl apply -k deploy/overlays/dev
```



Étape	Ce qui se passe	Si ça casse
1. Rendu	base + overlay assemblés sur le poste	erreur YAML : rien ne part
2. API server	vérifie et enregistre : created	unknown field : refus immédiat
3. Contrôleurs	Deployment, ReplicaSet, pods demandés	READY 0/1 au début
4. Admission	Pod Security, quota, LimitRange	aucun pod : FailedCreate
5. Scheduler	un node avec la place demandée	Pending
6. kubelet	conteneur, puis sondes ; prêt : dans le Service	ErrImagePull, CrashLoopBackOff

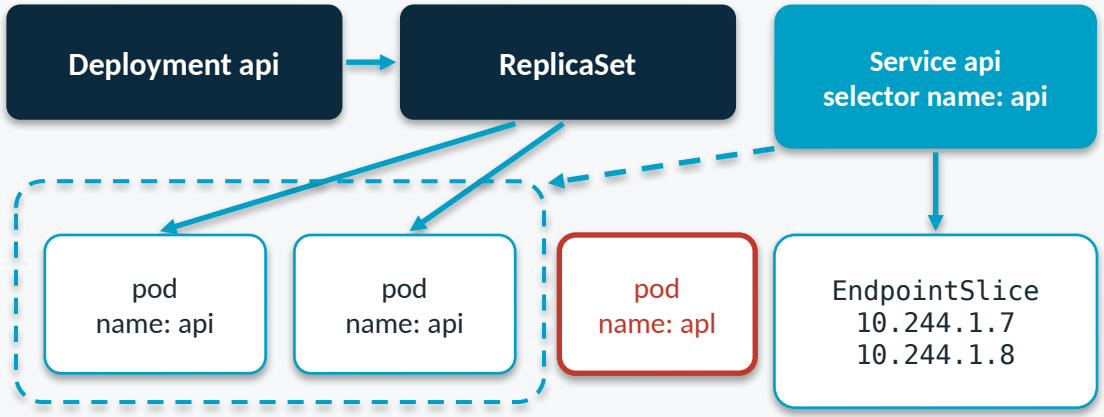
MODULE 5

Un Service, un Deployment, une étiquette

Rien ne relie les deux objets sauf l'étiquette : une faute de frappe, et plus rien ne passe

YAML - deploy/base/api.yaml (extrait)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: api                          # le nom joint par nginx
spec:
  selector:
    app.kubernetes.io/name: api   # choisit les pods
  ports:
    - {name: http, port: 8080, targetPort: http}
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: api}
spec:
  template:
    metadata:
      labels: {app.kubernetes.io/name: api}
```



1. Le Deployment pose l'étiquette sur chaque pod qu'il crée
2. Le Service cherche les pods qui portent l'étiquette
3. Seuls les pods prêts y entrent ; nginx joint ce nom : api
4. Une faute de frappe : invisible pour les deux, aucun message

MODULE 5

Le modèle api.yaml, bloc par bloc

Chaque bloc a une raison, et un symptôme quand il manque

Bloc	Ce qu'il fait	Ce qui casse sans lui
replicas, strategy	1 copie ; un pod neuf de plus, aucun ancien retiré avant	Recreate, ou maxUnavailable 1 : coupure
securityContext du pod	runAsNonRoot, runAsUser, seccompProfile	refusé par le namespace (restricted)
securityContext du conteneur	pas d'escalade, lecture seule, aucune capacité	idem
startupProbe	laisse 2 s x 30 au démarrage	un démarrage lent tué en boucle
readinessProbe	/readyz : prêt à recevoir du trafic ?	du trafic vers un pod qui ne répond pas
livenessProbe	/healthz : vivant ? sinon redémarré	un processus bloqué reste en place
resources	réserve et plafonne CPU et mémoire	refusé par le quota du namespace
volumes projected	les clés de plusieurs Secrets dans /run/secrets	le service sort en code 78
emptyDir sur /tmp	un dossier inscriptible malgré la lecture seule	plantage à la première écriture

MODULE 5

Ce que chaque service attend

Relevé dans les images et dans app/compose.yaml : c'est la matière de l'atelier

Service	uid:gid	Ports	Sondes	Ressources	Inscriptible
ingest	10001:10001	8081, 9100	/healthz, /readyz	50m, 96Mi / 500m, 192Mi	/tmp
nginx	101:101	8080, 8443	/healthz sur 8080	20m, 32Mi / 250m, 64Mi	/tmp
loadgen	10001:10001	9100	aucune	20m, 64Mi / 250m, 128Mi	aucun

Windows, macOS et Linux - le relever soi-même

```
docker image inspect velocite/nginx:2.0.0 --format "{{.Config.User}} {{.Config.ExposedPorts}}"
# puis, dans app/compose.yaml : les lignes secrets:, tmpfs et target: de chaque service
```

MODULE 5

La sécurité du pod, imposée par le namespace

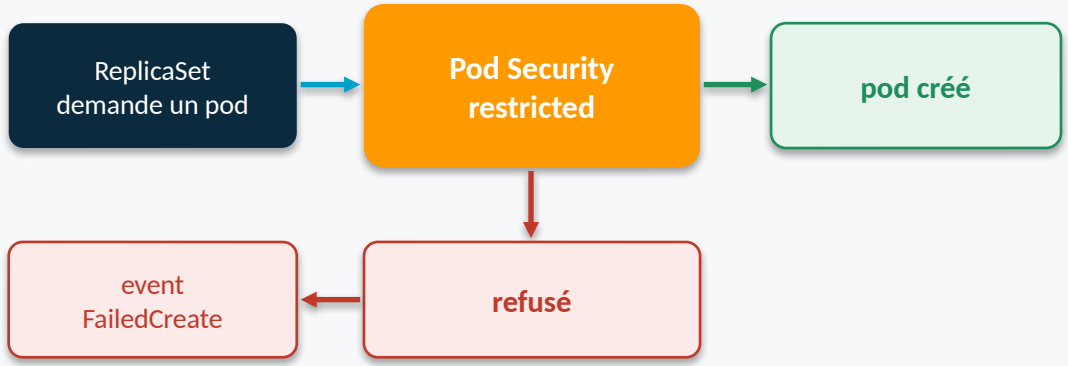
Mardi, le namespace refuse les pods trop privilégiés ; voici ce qu'il exige, et où se lit le refus

YAML - deux blocs securityContext

```yaml
spec:
  securityContext:                # le pod
    runAsNonRoot: true
    runAsUser: 10001              # l'uid de l'image
    runAsGroup: 10001
    seccompProfile: {type: RuntimeDefault}
  containers:
    - name: api
      securityContext:            # le conteneur
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: {drop: [ALL]}
```

1

2



1. Le pod : jamais root, l'uid de l'image, le filtre seccomp
2. Le conteneur : pas d'élévation, lecture seule, aucune capacité
3. Le namespace examine chaque pod avant de l'écrire
4. Un réglage manque : aucun pod, get pods vide ; lire get events

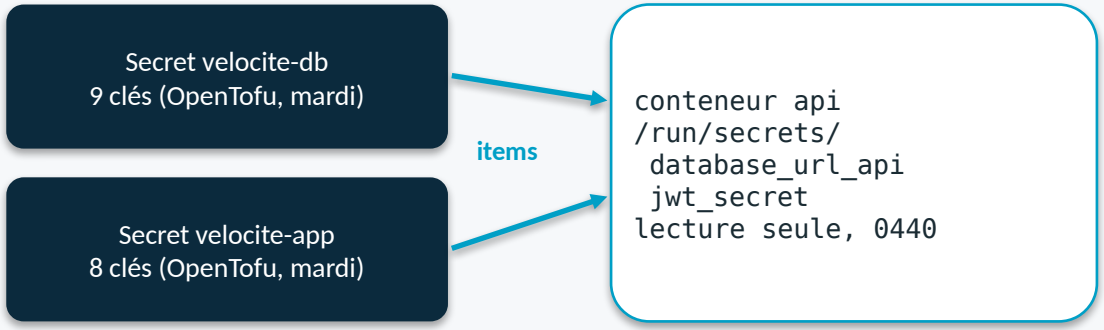
MODULE 5

Les secrets, montés en fichiers

Les mêmes noms qu'en Compose : /run/secrets/<nom>, lus par les services

YAML - un volume projected (extrait)

```yaml
volumes:
  - name: secrets
    projected:
      defaultMode: 0440
      sources:
        - secret:
            name: velocite-db
            items:
              - {key: database_url_api,
                 path: database_url_api}
        - secret:
            name: velocite-app
            items:
              - {key: jwt_secret, path: jwt_secret}
```



1. Les Secrets créés mardi, sous ces noms exacts
2. items : seulement les clés utiles, pas le mot de passe admin
3. Un fichier par clé, lu par le service comme en Compose
4. Clé absente du Secret : ContainerCreating.
   Item oublié : l'API sort en 78.

MODULE 5

Kustomize : une base, des overlays

Écrire une fois, corriger par environnement, sans copier

```
base/
  web.yaml
    Service web : ClusterIP
  api.yaml
    image: monimage
  kustomization.yaml
```

La base : ce qui est commun à tous les environnements. On ne l'applique jamais seule.

overlays/exemple/kustomization.yaml

```
1 namespace: exemple
  resources:
    - ../../base
2 images:
    - name: monimage
      newTag: 1.3.0
3 patches:
    - path: service.yaml
```

overlays/exemple/service.yaml

```
  apiVersion: v1
  kind: Service
3 metadata: {name: web}   # désigne l'objet
  spec:
    type: NodePort
    ports:
      - {port: 8443, nodePort: 30443}
```

1. namespace: exemple
   posé sur chaque objet
2. image: monimage:1.3.0
   la base n'a pas bougé
3. Service web : NodePort 30443
   le reste vient de la base
4. **kubectl kustomize :**
   le rendu, à lire avant d'appliquer

MODULE 5

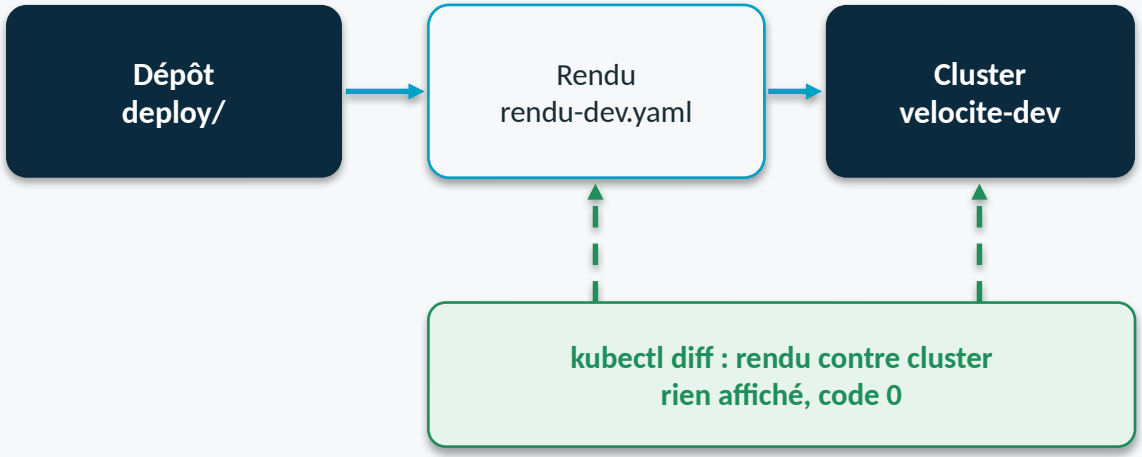
Regarder, appliquer, prouver

Le rendu avant, kubectl diff après : la même preuve d'idempotence que mardi

Windows, macOS et Linux

```
kubectl kustomize deploy/overlays/dev > rendu-dev.yaml
kubectl apply -k deploy/overlays/dev
kubectl diff -k deploy/overlays/dev
# rien affiché, code 0 : dépôt et cluster d'accord
# Windows : le diff de Git (Commandes_J3)
```

1. Le rendu se lit dans l'éditeur ; il n'est jamais versionné
2. apply envoie : created, configured, unchanged
3. Code 1 et des lignes - et + : un écart à lire

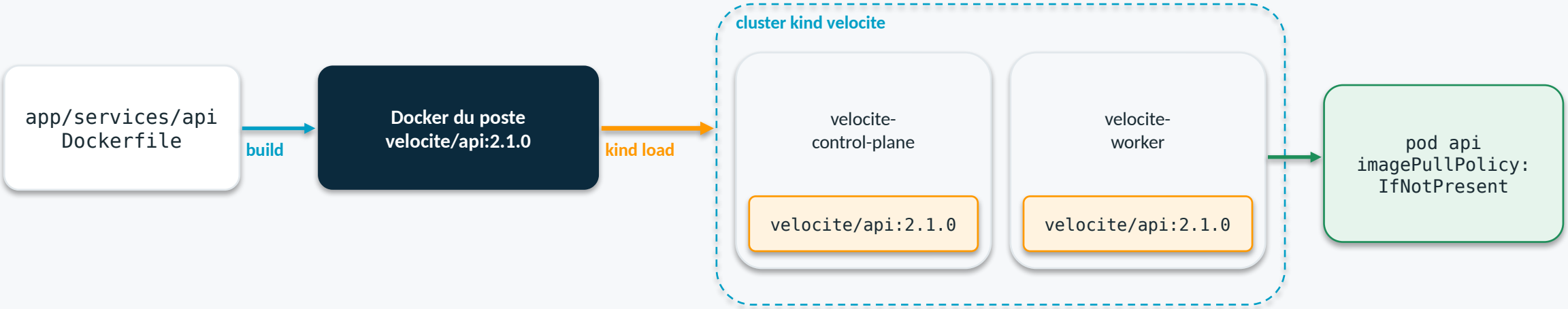


4. Un détail qui trompe : au second apply, postgres, garage et les budgets d'interruption affichent configured. Le serveur ajoute des champs, rien ne change. La preuve, c'est le diff vide, pas le mot affiché.

MODULE 5

Les images dans kind

Les nodes ne voient pas les images de votre Docker : on les y copie



1. charger-images.ps1 construit les neuf images avec Docker, sur le poste
2. Chaque node est un conteneur, avec son propre stock d'images, vide au départ
3. kind load copie chaque image dans chaque node : une à deux minutes
4. IfNotPresent : le node prend l'image qu'il a ; sans elle, ErrImagePull

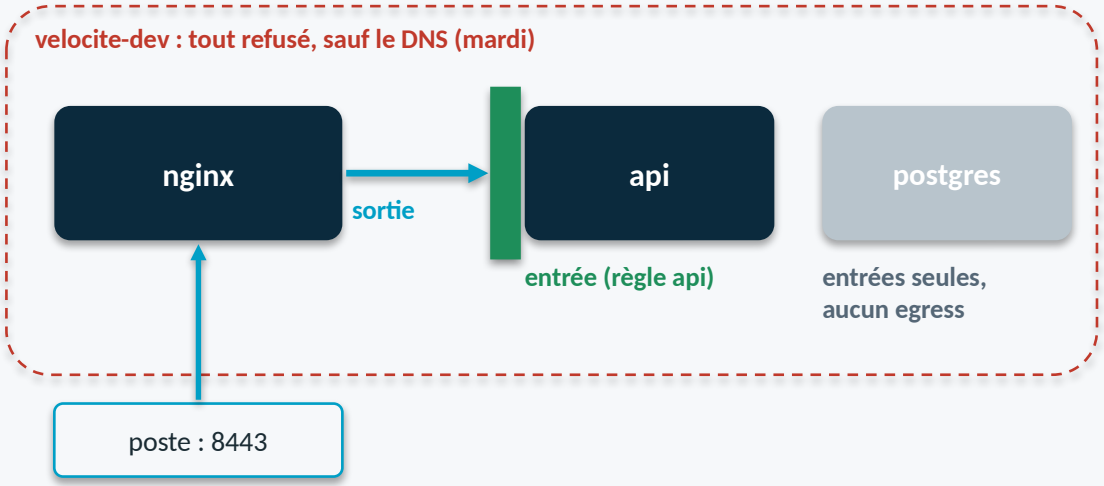
MODULE 5

Une règle réseau, sur le modèle de nginx

Qui entre, vers qui il sort : un flux s'ouvre des deux côtés

YAML - deploy/base/reseau.yaml (fourni)

```yaml
kind: NetworkPolicy
metadata: {name: nginx}
spec:
  podSelector:                                  # 1
    matchLabels: {app.kubernetes.io/name: nginx}
  policyTypes: [Ingress, Egress]
  ingress:                                      # 4
    - ports:                    # le poste (port de noeud)
        - {port: 8443, protocol: TCP}
  egress:                                       # 2
    - to:
        - podSelector:
            matchLabels: {app.kubernetes.io/name: api}
      ports:
        - {port: 8080, protocol: TCP}
```



1. Une règle choisit ses pods, puis déclare ses deux sens
2. La sortie chez nginx seule : la connexion attend, puis expire
3. L'entrée chez api, depuis nginx : le flux passe
4. Le poste entre par le port de noeud : on ne filtre que le port
5. Une base n'appelle personne : aucun bloc egress

ATELIER 5

Atelier 5 - VéloCité dans l'environnement dev

95 min - en binôme - la fiche Fiche_Atelier5_J3 (Moodle) : ce qui est attendu, où trouver un exemple

Consignes

- Chargez les images (charger-images.ps1)
- ingest.yaml, nginx.yaml, loadgen.yaml sur le modèle d'api.yaml ; ajoutez-les à la base
- Dans reseau.yaml, les règles d'ingest, loadgen, postgres et batch
- Overlay dev : frontal en NodePort 30443, VELOCITE_ENV=dev
- Le rendu, puis apply ; attendez que tout soit prêt
- smoke.ps1 -Env dev : tout en OK ; kubectl diff vide
- DEPLOIEMENT.md section 5 ; commit, push
- Fini avant l'heure : la fiche, Si vous avez fini (solo)

Windows, macOS et Linux - depuis $DEPOT

```
./scripts/charger-images.ps1
kubectl kustomize deploy/overlays/dev > rendu-dev.yaml
kubectl apply -k deploy/overlays/dev
kubectl -n velocite-dev get pods -w   # Ctrl+C
kubectl -n velocite-dev describe pod NOM_DU_POD
kubectl -n velocite-dev get events
./scripts/smoke.ps1 -Env dev
kubectl diff -k deploy/overlays/dev
# macOS, Linux : pwsh devant chaque .ps1
# smoke.ps1 : la phrase du state requise
# NOM_DU_POD : un nom lu dans get pods
```

ATELIER 5

Si ça coince

Les blocages relevés pendant la répétition, et leur sortie

Ce qu'on voit	La cause	La sortie
Le Deployment existe, aucun pod	refusé par Pod Security	get events : le réglage manquant est nommé
CrashLoopBackOff, code 78 ou 1	un fichier de secret absent	logs --previous le nomme ; items et compose.yaml
Exit Code 75	une dépendance injoignable	règle réseau, ou postgres pas encore prêt
504 sur https://localhost:8443	règle réseau d'un seul côté	sortie chez l'appelant ET entrée chez l'appelé
ErrImagePull, ImagePullBackOff	image non chargée dans kind	relancer charger-images.ps1
Plantage à la première écriture	pas de /tmp inscriptible	emptyDir monté sur /tmp
kubectl diff : executable file not found	pas de diff (Windows)	Commandes_J3 : le diff de Git

Ce n'est pas le DNS.
Ça ne peut pas être le DNS.
C'était le DNS.

Le haïku de l'exploitation. Un nom qui ne se résout plus, une connexion qui expire : vérifiez d'abord la règle DNS, puis les deux côtés du flux.

ATELIER 5

Le point de midi

Cinq minutes : où en êtes-vous ? Cet après-midi construit sur votre dev

Avant de déjeuner

- Commit et push de ce qui est écrit, même incomplet
- kubectl -n velocite-dev get pods : notez ce qui ne tourne pas
- Le cluster peut rester allumé pendant le déjeuner

À 13h40, il faut au minimum

- api, postgres-0, redis, garage-0 en Running
- nginx en Running : https://localhost:8443 répond
- Sinon : un état de reprise existe, demandez-le-moi à 13h40
- ingest et loadgen peuvent se finir pendant l'atelier 6

MODULE 6

Mettre à jour sans couper, revenir en arrière

- Le mécanisme d'une mise à jour progressive, et quand le neuf ne démarre pas
- Diagnostiquer une version qui ne démarre pas, sans deviner
- Revenir en arrière en une minute, puis réparer par le dépôt

MODULE 6

Les mots d'abord

Sept mots pour les mises à jour

Mot	Ce que c'est
Rolling update	Remplacer les pods un par un ; l'ancienne version sert pendant ce temps
maxSurge / maxUnavailable	Combien de pods en plus / en moins on tolère pendant la mise à jour
progressDeadlineSeconds	Au-delà, la mise à jour est déclarée bloquée ; elle n'est pas annulée
Rollout	Une mise à jour d'un Deployment : rollout status, history, undo
HPA	HorizontalPodAutoscaler : ajuste le nombre de copies selon la charge
PodDisruptionBudget	Empêche une opération volontaire de retirer le dernier pod prêt
Promotion	Passer en prod une version qui a tourné en dev : changer un newTag

MODULE 6

Le mécanisme d'une mise à jour

maxSurge 1, maxUnavailable 0, une copie : l'ancien ne part que quand le neuf est prêt

YAML - deploy/base/api.yaml

```
  replicas: 1
  strategy:
    rollingUpdate:
      maxSurge: 1          # un pod en plus
      maxUnavailable: 0    # aucun en moins
```

2 4

5 Ce qui casse : maxUnavailable: 1 sur une seule copie. L'ancien part d'abord ; le service est coupé pendant tout le démarrage, et pour de bon si la nouvelle version ne démarre pas.



Un pod sert les clients

Un pod de plus : deux pods

La readinessProbe répond : il reçoit du trafic

Seulement alors, l'ancien est retiré

MODULE 6

Quand le neuf ne démarre jamais

La mise à jour bloque ; l'ancien reste ; le service tient

Windows, macOS et Linux

```
kubectl -n velocite-dev rollout status deployment/api
Waiting for deployment "api" rollout to finish: 1 old replicas are pending termination...
error: deployment "api" exceeded its progress deadline
```



1 t1 : le neuf démarre, plante

v1 prête | v2 plante

Il redémarre en boucle : CrashLoopBackOff

2 t2 : il n'est jamais prêt

v1 prête | v2 plante

L'ancien reste : les clients reçoivent l'ancienne version

3 t3 : 120 s plus tard

v1 prête | v2 plante

exceeded its progress deadline : bloquée, pas annulée

Le piège : "old replicas are pending termination" parle des anciens, qui attendent. C'est le neuf qui bloque ; les anciens servent.

MODULE 6

Diagnostiquer sans deviner

Cinq questions, dans cet ordre - toutes les commandes avec -n velocite-dev

	La commande	La question	Où regarder
1	`kubectl rollout status deployment/api`	**La mise à jour est-elle bloquée ?**	exceeded its progress deadline
2	`kubectl get pods`	**Qui va mal ?**	STATUS, RESTARTS
3	`kubectl describe pod NOM_DU_POD`	**Pourquoi s'est-il arrêté ?**	Last State : Exit Code (app/README.md)
4	`kubectl logs NOM_DU_POD --previous`	**Qu'a-t-il dit avant de s'arrêter ?**	les dernières lignes
5	`kubectl rollout history deployment/api app/CHANGELOG.md`	**Qu'est-ce qui a changé ?**	la version, et ce qu'elle exige

--previous : les logs du conteneur qui a planté, pas de celui qui vient de redémarrer.

MODULE 6

Rétablir tout de suite, réparer durablement

Deux gestes différents, dans cet ordre - kubectl diff dit ce qui reste à faire

	Le geste	Le dépôt (overlay dev)	Le cluster	kubectl diff
	`apply -k (2.2.0)`	newTag: 2.2.0	v2 plante, v1 sert	vide : le dépôt a voulu ça
1	`rollout undo`	newTag: 2.2.0 (inchangé)	v1 seule, sert	NON VIDE : code 1
2	**Le prochain apply, ou le prochain up.ps1, remettrait la panne.**			
3	`corriger, apply -k`	newTag: 2.2.0 + le manifeste corrigé	v2 prête, sert	vide : code 0

4 Jamais kubectl edit ni kubectl set env : perdu au prochain apply, comme la dérive de mardi. Avant toute mise à jour : lire app/CHANGELOG.md.







MODULE 6

La production : plus de copies et ce qui va avec

Un autoscaler, un budget d'interruption, et un piège entre les deux

YAML - exemples neutres

```yaml
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: {kind: Deployment, name: web}
  minReplicas: 2
  maxReplicas: 4
  metrics:
    - type: Resource
      resource: {name: cpu, target:
        {type: Utilization, averageUtilization: 70}}
# overlay prod : l'autoscaler décide seul
patches:
  - target: {kind: Deployment, name: web}
    patch: |-
      - op: remove
        path: /spec/replicas
```

YAML - un budget d'interruption

```yaml
kind: PodDisruptionBudget
spec:
  minAvailable: 1
  selector:
    matchLabels: {app.kubernetes.io/name: web}
```

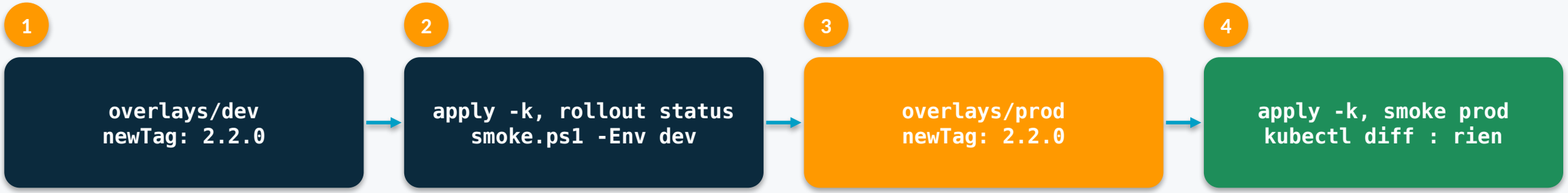
Le piège

Si le Deployment fixe aussi replicas, chaque apply remet ce nombre et défait l'autoscaler : l'overlay prod retire replicas de l'API.

MODULE 6

La promotion : dev d'abord, prod ensuite

Une version n'arrive en prod qu'après avoir tourné en dev



Le tag de dev change d'abord

La version tourne, les tests passent

Puis, seulement, le tag de prod

La prod est conforme au dépôt

Ce qui le rend possible

- Une base commune : dev et prod ne diffèrent que par l'overlay ; les mêmes images, chargées une fois
- Le tag de l'overlay prod est le seul endroit où l'on choisit la version de prod

ATELIER 6

Atelier 6, partie 1 - La prod

30 min avant la pause, à finir après - en binôme - la fiche Fiche_Atelier6_J3 (Moodle)

Consignes

- infra/20-env : workspace prod, envs/prod.tfvars
- Overlay prod : namespace velocite-prod, frontal en NodePort 30444, VELOCITE_ENV=prod
- Deux copies d'ingest et de nginx ; l'autoscaler de l'API (2 à 4)
- Un budget d'interruption pour api, ingest et nginx
- Appliquez, puis smoke.ps1 -Env prod

Windows, macOS et Linux - depuis $DEPOT

```
tofu -chdir=infra/20-env workspace select -or-create prod
tofu -chdir=infra/20-env apply -var-file envs/prod.tfvars
tofu -chdir=infra/20-env workspace select dev
kubectl kustomize deploy/overlays/prod > rendu-prod.yaml
kubectl apply -k deploy/overlays/prod
kubectl -n velocite-prod get pods,hpa,pdb
./scripts/smoke.ps1 -Env prod
# macOS, Linux : pwsh ./scripts/smoke.ps1 -Env prod
# la prod : https://localhost:9443
```

ATELIER 6

Atelier 6, partie 2 - Une nouvelle version

90 min après la pause, dont 15 pour finir la prod - en binôme - pièces : la réparation dans le dépôt, la section Retour arrière de RUNBOOK.md

Consignes

- L'équipe de développement livre l'API 2.2.0 : passez dev en 2.2.0 (newTag), appliquez, suivez
- Quelque chose ne va pas. Le service est-il coupé ?
- Trouvez la cause avec les cinq questions du module 6
- Rétablissez le service tout de suite
- Réparez par le dépôt, redéployez la 2.2.0 en dev, puis promouvez
- RUNBOOK.md, Retour arrière : ce qu'on voit, vérifier, agir, confirmer
- ARCHITECTURE.md : Les deux environnements, Ce qui garde la plateforme durcie

Windows, macOS et Linux - depuis $DEPOT

```
kubectl apply -k deploy/overlays/dev
kubectl -n velocite-dev rollout status deployment/api
kubectl -n velocite-dev get pods
./scripts/smoke.ps1 -Env dev -PendantIncident
# les clients sont-ils touchés ? le premier contrôle répond
kubectl -n velocite-dev logs NOM_DU_POD --previous
kubectl -n velocite-dev rollout undo deployment/api
kubectl diff -k deploy/overlays/dev
# macOS, Linux : pwsh devant ./scripts/smoke.ps1
# NOM_DU_POD : un nom lu dans get pods
```

ATELIER 6

Si ça coince

Les blocages relevés pendant la répétition, et leur sortie

Ce qu'on voit	La cause	La sortie
rollout status : exceeded its progress deadline	le neuf ne devient jamais prêt	get pods, puis describe et logs --previous
old replicas are pending termination	les anciens attendent un neuf prêt	ils servent toujours : smoke -PendantIncident
smoke : un contrôle KO pendant l'incident	le pod neuf plante, c'est attendu	-PendantIncident saute ce seul contrôle
Après undo, kubectl diff non vide	le dépôt dit encore la nouvelle version	c'est attendu : réparer le dépôt
Réparé, mais /api/version dit l'ancienne	l'ancien pod finit ses requêtes	attendre la fin de rollout status
Un pod de prod manque (READY 1/2)	quota de prod : FailedCreate	get events ; le quota est dans envs/prod.tfvars
undo : Warning last-applied-configuration	undo ne touche pas au dépôt	attendu : réparer le dépôt, puis apply

BILAN

Ce que vous avez construit depuis lundi

Trois jours, quatre étages, et pour chacun une preuve qui se rejoue

LUNDI	MARDI	MERCREDI MATIN	MERCREDI APRÈS-MIDI
Des images sûres - L'inventaire de la plateforme - L'image du batch : multi-stage, digest, non root - La CI : build, Trivy, utilisateur	**L'infrastructure en code** - OpenTofu : plan, apply, state - Le cluster kind et ses outils - dev : un namespace fermé, des secrets générés, un state chiffré	**VéloCité sur Kubernetes** - Des manifestes, sur le modèle d'api.yaml - Une règle réseau par service - Kustomize : une base, l'overlay dev	**La prod, et une version qui casse** - L'overlay prod : copies, autoscaler, budgets - Un rollout bloqué, un service qui tient, undo - La réparation par le dépôt, la promotion, le runbook
Preuve : la CI verte sur main	Preuve : tofu plan : No changes ; le pod root refusé	Preuve : smoke dev vert ; kubectl diff vide	Preuve : smoke prod vert ; aucun client coupé

BILAN

La preuve du jour

Ce que vous savez prouver ce soir, et à qui

Windows, macOS et Linux

```
./scripts/smoke.ps1 -Env dev
./scripts/smoke.ps1 -Env prod
# Tous les contrôles sont passés.
kubectl diff -k deploy/overlays/dev
kubectl diff -k deploy/overlays/prod
# rien : le dépôt décrit exactement
# ce qui tourne
git add -A
git commit -m "Atelier 6 : prod, 2.2.0, runbook"
git push
# macOS, Linux : pwsh devant les scripts
```

En une phrase, pour la directrice financière

"Une nouvelle version qui ne marche pas n'a coupé le service pour aucun client. On est revenu à la version d'avant en une minute, puis on a corrigé la vraie cause pour que ça ne se reproduise pas."



Version 10.17 : le processeur ne chauffe plus quand on maintient la barre d'espace. Un utilisateur : "Cette mise à jour casse mon usage, je m'en servais comme touche Ctrl." Toute correction casse l'habitude de quelqu'un : d'où le CHANGELOG, lu avant chaque mise à jour.

xkcd 1172, Workflow - Randall Munroe, CC BY-NC 2.5

BILAN

Demain : le bilan quotidien, et la supervision

Jeudi 22 octobre - un pipeline qui décide seul, des alertes qui préviennent

Ce qui arrive

- Le job batch de lundi devient un workflow Argo, planifié
- Relances, arrêt au premier échec, issue publiée
- Des alertes, leurs tests, un tableau de bord

Ce soir, cinq minutes

- Commit et push, chez les deux membres du binôme
- Le cluster peut rester en marche : demain le reprend
- Relire votre section Retour arrière : la suivriez-vous sans vous ?